Software Engineering Report: ResolvIQ

1. Introduction

ResolvIQ is an end-to-end complaint management program built to help organizations receive, process, and resolve client concerns through a structured digital workflow. The program integrates a role-based frontend, a backend service layer, and a relational database to manage the full complaint lifecycle from submission to resolution and feedback.

The system supports three primary roles: clients who submit and track complaints, staff who work on assigned cases, and administrators who oversee assignment, monitoring, and governance. By combining operational workflows, communication channels, status tracking, and reporting-ready data structures, the program is designed to improve transparency, accountability, and service quality across the organization.

2. Problem Description

Write the problem description

Organizations - including businesses, institutions, and government agencies - frequently struggle to manage complaints and issues efficiently. When clients or users encounter problems, they often face unclear reporting channels, long resolution times, and lack of visibility into the status of their submitted concerns.

Without a centralized, role-based system, complaints may be lost, misrouted, or left unresolved, leading to client dissatisfaction and reduced operational efficiency. Staff members lack structured workflows for addressing issues, and administrators have limited tools for monitoring, prioritizing, and assigning cases appropriately.

3. Stakeholders

Identify stakeholders

User/clients - are the primary initiators of complaints and issue reports. They interact with the system to seek assistance and resolution for problems they have encountered.

Staff - members are responsible for responding to and resolving complaints assigned to them by administrators. They serve as the frontline agents for client issue resolution.

Admin - hold the highest level of access and are responsible for overseeing the entire complaint management workflow, ensuring that issues are properly handled, and maintaining system operations.

4. System Requirements

Write system requirements (functional and non-functional)

Functional

User Authentication/Login - The system shall provide a secure login mechanism for all three roles - Users/Clients, Staff, and Admin. Each role shall have a unique dashboard and access level upon successful authentication. The system shall support password recovery and session management.

Complaint Status Tracking - Clients shall be able to view the real-time status of all their submitted complaints through a personalized dashboard. A timeline view shall display the history of status changes and staff actions taken on each complaint.

Clients posting complains - Clients shall be able to submit new complaints by filling out a structured form including: complaint title, description, category, priority level, and optional file attachments. Each submitted complaint shall receive a unique tracking ID upon creation.

Admin assigning post complains - The admin shall be able to view all submitted complaints and manually assign each complaint to a specific staff member based on category or expertise. The system shall send an automatic notification to the assigned staff member upon assignment.

Admin Reporting and Analytics - Admins shall have access to a reporting module that generates summaries of complaint volumes, resolution rates, average resolution times, staff performance, and client satisfaction ratings, with filtering by date range and category.

Staff solving complains - Staff members shall be able to view their assigned complaints, update resolution statuses (Pending, In Progress, Resolved, Escalated), add internal notes, and communicate resolution steps to the client. Resolved complaints shall prompt a closure confirmation.

Real-time Notifications - The system shall send real-time in-app notifications to all relevant parties upon key events, including: complaint submission, assignment, status update, escalation, and resolution. Email notifications shall also be supported as a secondary channel.

Rating System - Upon complaint resolution, clients shall be prompted to rate the service quality on a scale of 1 to 5 stars and provide optional written feedback. Ratings shall be visible to the admin on the performance dashboard and shall contribute to staff performance metrics.

Email Notifications (via Postmark API) - The system shall use the Postmark API as the transactional email delivery service, as the hosting environment (Render) does not support direct SMTP connections. All email notifications shall be sent through Postmark's HTTP API to ensure reliable and trackable delivery. The following email notification triggers are supported:

Staff Application Submitted - When a new user registers as a Staff Member, the system shall automatically send a confirmation email to the applicant acknowledging that their application has been received and is pending administrator review. The admin shall also receive a notification email alerting them of a new pending staff application.

Staff Application Approved - When an administrator approves a staff account application, the system shall send an approval email to the applicant notifying them that their account has been activated. The email shall include a direct link to the login page and their assigned specialization(s).

Staff Application Rejected - When an administrator rejects a staff account application, the system shall send a rejection email to the applicant informing them that their application was not approved. The email may optionally include a reason for rejection as provided by the admin.

Non-functional

Performance - All messages sent within the system - including complaint status updates, staff-to-client communications, and notifications - shall be delivered within 2 to 3 seconds under normal operating conditions to ensure a responsive and seamless user experience.

Availability - The system shall maintain a minimum uptime of 99.5% per month, ensuring uninterrupted access for all user roles. Scheduled maintenance windows shall be communicated in advance and minimized to off-peak hours.

Scalability - The system shall be capable of supporting up to 10,000 concurrent users without degradation in performance. The architecture shall support horizontal scaling to accommodate future growth in user base and data volume.

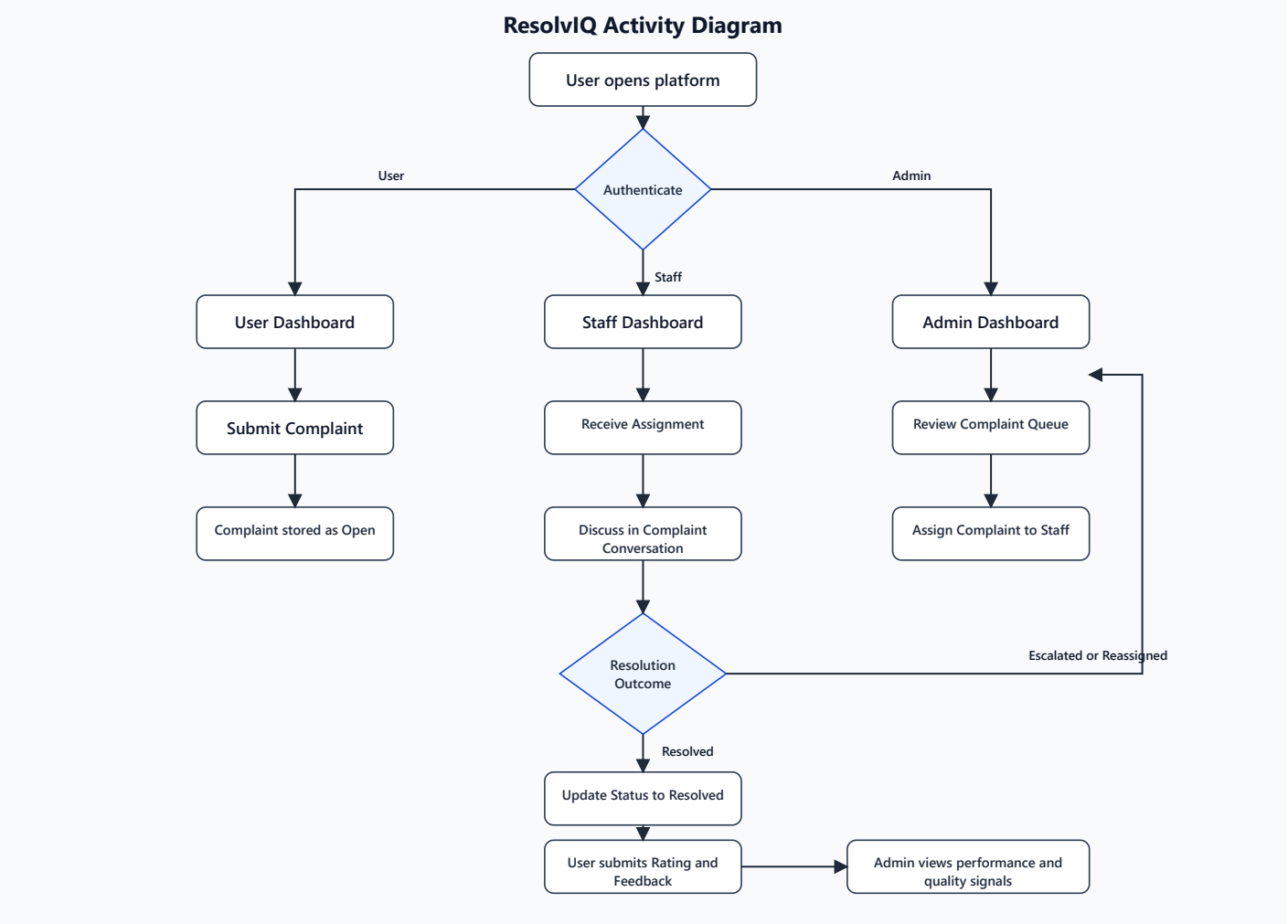
Security - All user data and communications shall be encrypted in transit using HTTPS/TLS protocols. Role-based access control (RBAC) shall ensure that users only access data and features permitted for their role. The system shall enforce strong password policies and support multi-factor authentication (MFA).

Usability - The interface shall be intuitive and accessible to non-technical users. The system shall comply with WCAG 2.1 Level AA accessibility standards. Mobile responsiveness shall be supported across common screen sizes and devices.

5. System Modeling

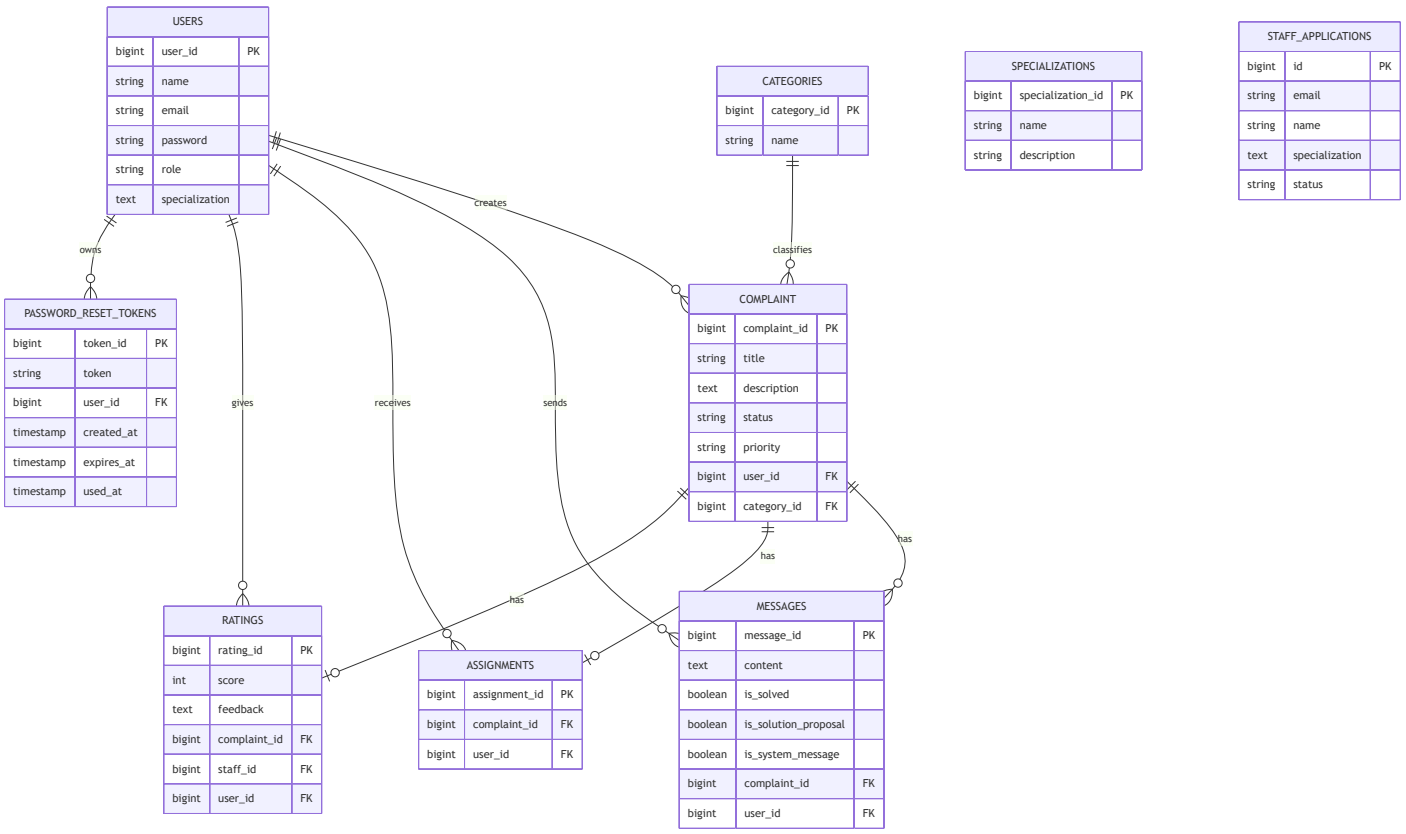
Use Case Diagram

Activity Diagram



6. Database Design

ER Diagram

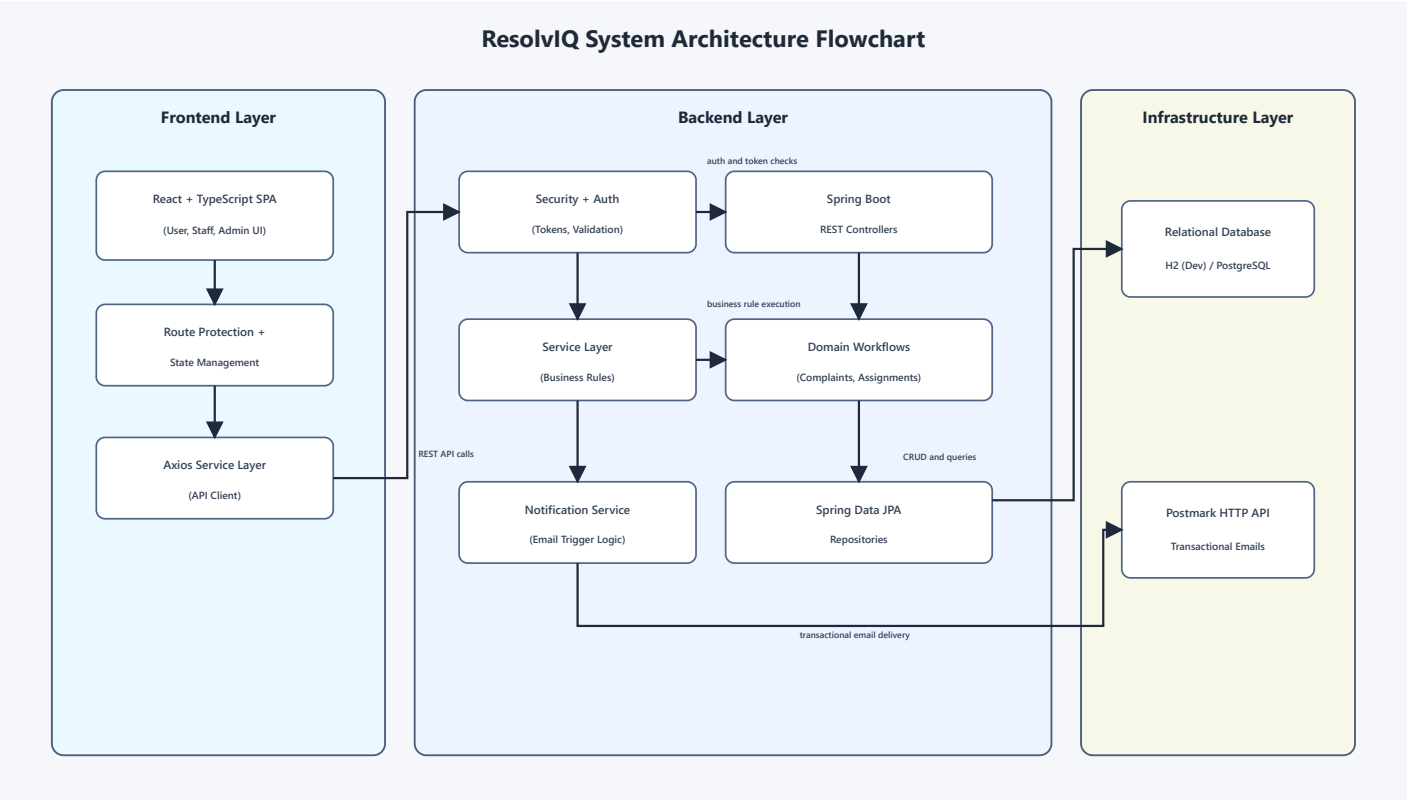


Database Tables

- `users`: account records for user, staff, and admin roles
- `complaint`: complaint lifecycle records and metadata
- `messages`: complaint conversation entries and system messages
- `assignments`: complaint-to-staff assignment mapping
- `ratings`: user feedback for complaint outcomes
- `categories`: complaint category reference data
- `specializations`: specialization options for staff workflows
- `staff_applications`: pending/approved/rejected staff onboarding requests
- `password_reset_tokens`: one-time password reset tokens

7. System Architecture

ResolvIQ follows a layered client-server architecture.



Structure summary:

- Frontend handles role-based UI, forms, dashboards, and route protection.
- Backend controller layer exposes endpoints for users, complaints, messages, assignments, ratings, categories, specializations, and staff applications.
- Service layer applies validation and business rules.
- Repository layer persists entities through JPA.
- Database stores relational records with entity links.

8. System Implementation

The system was built as an integrated web application with the following implementation approach:

- Backend implementation:
  - Java Spring Boot services for complaint workflows
  - JPA entity mapping for relational persistence
  - REST controllers for CRUD and role workflows
  - token issuance and password reset flows
  - transactional emails using Postmark-compatible delivery
- Frontend implementation:
  - React + TypeScript single-page application
  - Axios API client with request/response interceptors
  - role-based dashboards (user, staff, admin)
  - service modules for complaints, assignments, messaging, ratings, categories, and authentication
- Operational workflows implemented:
  - complaint creation, assignment, transfer, and status updates
  - threaded communication between user and staff
  - staff onboarding approval workflow
  - rating capture after complaint resolution

9. System Testing

Testing Methods Used

- Automated backend smoke testing through Spring Boot test context loading.
- Source-level test inventory review for backend and frontend test coverage.
- Manual workflow verification through integrated API + UI paths.

Testing Results

- Backend automated tests currently include one context-load test file:
  - `backend/src/test/java/com/rbcits/backend/BackendApplicationTests.java`
- Frontend automated test files were not found in the current source tree.
- Integration behavior is implemented across backend endpoints and frontend service modules (complaints, messages, assignments, ratings, auth, and staff applications).

10. Conclusion

ResolvIQ addresses a real complaint-management problem through a role-based, end-to-end system that unifies issue submission, assignment, communication, and feedback.

Learned outcomes by member role:

- UI/UX: Designing role-specific dashboards improved understanding of user journey clarity, visual hierarchy, and interaction consistency for user, staff, and admin interfaces.

- Database Design: Modeling entity relationships for complaints, assignments, messages, ratings, and staff applications strengthened normalization practice and lifecycle-driven schema planning.
- Documentation: Maintaining synchronized technical documentation reinforced the importance of traceability between requirements, architecture, implementation, and testing evidence.
- Software Development: Implementing integrated frontend-backend workflows improved capability in API contract alignment, service-layer validation, and end-to-end feature delivery.

Possible improvements:

- add stricter backend authorization enforcement per endpoint and role
- increase automated test coverage for services and frontend behavior
- add formal performance/load testing against target concurrency
- implement richer analytics and reporting dashboards
- add complaint attachment support with secure file handling